

추론 파이프라인: proposal_planning → CodeAgent → 최종 답변

작성일: 2026-04-25

최종 수정: 2026-04-25 (버킷 검색 추가, answer_plan 흐름 명확화)

대상 코드: `examples/open_deep_research/`

사용 언어: Python 3 / smolagents / sentence-transformers / TogetherAI

1. 전체 흐름 개요

```
run_gaia.py → answer_single_question()
|
├─ [입력] example = { question, file_name, true_answer, ... }
|
├─ [KB 조회] AKBCClient → odysseuss_db.jsonl
|   retrieval_type: hybrid / text / semantic / bucket
|   ↑
|   bucket 검색 (신규):
|       Stage 1 - normalized task_type × domain 버킷 후보 수집
|       Stage 2 - 후보 내 TF-IDF 텍스트 랭킹 → top_k 선정
|
├─ [proposal_planning()] planner_kb/planner.py
|   Stage 1a generate_knowledge → knowledge_draft
|   Stage 1b refine_knowledge → knowledge_text (Decision Guide 있을 때만)
|   Stage 2a generate_plan → plan_draft
|   Stage 2b refine_plan → plan_str (Decision Guide 있을 때만)
|   Stage 3a generate_instance → instance_draft
|   Stage 3b refine_instance → instance_str (Decision Guide 있을 때만)
|   반환: { "plan": "Knowledge:\n...\n\nPlan:\n...\n\nInstance:\n..." }
|
├─ [CodeAgent.planning_step()] smolagents/agents.py
|   planner_fn(task, tools, managed_agents) 호출
|   → result["plan"] 을 answer_plan 으로 직접 사용
|   → PlanningStep 메모리에 기록
|
├─ [CodeAgent ReAct Loop]
|   Thought → Code → Observation (max_steps 회 반복)
|   Tools: Search, Crawl, TextInspect, AudioInspect, VisualInspect
|
└─ [prepare_response()] scripts/reformulator.py
    agent 메모리 → 단답형 최종 답변
```

주요 모듈

모듈	역할
<code>run_gaia.py</code>	전체 오케스트레이터, <code>answer_single_question()</code> 진입점

모듈	역할
agent_kb/ agent_kb_retrieval.py	KB 인덱싱 및 검색 (AgenticKnowledgeBase , AKB_Manager)
agent_kb/agent_kb_service.py	FastAPI 서비스 (포트 8005)
planner_kb/planner.py	proposal_planning() — 7단계 생성·교정
planner_kb/ planner_prompts.yaml	프롬프트 템플릿
smolagents/agents.py	CodeAgent.planning_step() — answer_plan 주입
scripts/reformulator.py	최종 답변 추출

2. KB 조회 단계

2.1 데이터 소스 — odyseuss_db.jsonl

각 레코드는 과거 태스크 경험 하나를 표현합니다.

필드	타입	설명
task_id	str	레코드 고유 ID
task	str	태스크 질문 텍스트
true_answer	str	정답 레이블
task_analysis	dict	{ knowledge, plan, task_type, domain, task_type_normalized, domain_normalized }
agent_planning	str/ null	에이전트가 생성한 원시 계획
decision_augmentation	dict	{ final_reference: {knowledge, plan, instance}, signals_summary: [...] }

로딩 시 정규화 처리 (parse_json_file):

```
# odyseuss_db: task_type / domain 이 {"raw": "...", "normalized": "..."} 형태
if isinstance(ta_type, dict):
    # 검색용: raw + normalized 모두 보관
    task_analysis["task_type"] = [ta_type.get("raw"),
    ta_type.get("normalized")]
    # 버킷 인덱싱용: normalized 값만 별도 저장
    task_analysis["task_type_normalized"] = [ta_type["normalized"]]
else:
    task_analysis["task_type_normalized"] = list(ta_type or [])

# domain 동일하게 처리
task_analysis["domain_normalized"] = ...

# decision_augmentation.final_reference 의 knowledge/plan 주입
if not task_analysis.get("knowledge") and fr.get("knowledge"):
    task_analysis["knowledge"] = fr["knowledge"]
```

2.2 검색 인덱스 초기화 순서

서비스 시작 시 `finalize_index()` 가 다음 순서로 인덱스를 빌드합니다.

1. `build_tfidf_indices()` - task 텍스트 TF-IDF 행렬 생성
2. `build_embeddings()` - all-MiniLM-L6-v2 sentence embedding 생성
3. `build_bucket_index()` - normalized task_type × domain 역인덱스 생성

2.3 검색 모드

bucket_rank_search (신규 — 2단계 검색)

정규화된 task_type과 domain을 기준으로 버킷 후보를 먼저 수집한 뒤, 그 안에서 텍스트 유사도로 최종 k개를 선정합니다.

Stage 1 - 버킷 후보 수집
query_types × query_domains 의 모든 (task_type, domain) 쌍에 대해
bucket_index[task_type][domain] 조회 → 후보 task_id 집합 구성
(각 버킷에 최소 1개 보장, 버킷별 10개 문서 사전 구축 예정)

Stage 2 - TF-IDF 텍스트 랭킹
후보 집합 내에서만 TF-IDF cosine similarity 계산
→ 상위 top_k 반환

후보 없으면 hybrid_search 로 fallback

hybrid_search

score = 0.5 × TF-IDF cosine + 0.5 × all-MiniLM-L6-v2 cosine

search_by_text

TF-IDF 벡터라이저로 task 필드 검색 (전체 문서 대상)

search_by_semantic

sentence-transformers 임베딩 후 cosine 검색 (전체 문서 대상)

type_domain_text_search

final_score = 0.3 × type_score + 0.3 × domain_score + 0.4 × text_score
type_score = |query_types ∩ kb_types| / max(|query_types|, 1)
domain_score = |query_domains ∩ kb_domains| / max(|query_domains|, 1)

2.4 버킷 인덱스 구조

```
# AgenticKnowledgeBase.bucket_index
{
  "Algebra and Operations": {
    "Mathematics General": ["gsm8k_001", "gsm8k_002", ...],
    "Finance Math":        ["mathqa_100", ...],
  },
  "Reasoning Skills": {
    "Mathematics General": [...],
  },
  ...
}
# 22개 normalized task_type × 40개 normalized domain
```

2.5 서비스 엔드포인트 (포트 8005)

엔드포인트	방식	설명
/search/bucket	POST	버킷 2단계 검색 (신규)
/search/hybrid	POST	TF-IDF + semantic 혼합
/search/text	POST	TF-IDF 전체 검색
/search/semantic	POST	semantic 전체 검색
/search/type_domain_text	POST	type·domain 가중 점수 검색

/search/bucket 요청 스키마:

```
{
  "query":      str,          # 질문 텍스트
  "task_types": list[str],     # normalized task_type 값들
  "domains":    list[str],     # normalized domain 값들
  "top_k":      int = 3
}
```

2.6 검색 결과 스키마

```
{
  "task_id":      str,
  "task":         str,
  "true_answer":  str,
  "task_analysis": {
    "knowledge":   str,
    "plan":        list[str],
    "task_type":   list[str], # raw + normalized
    "domain":      list[str],
    "task_type_normalized": list[str], # 버킷 인덱싱용
    "domain_normalized": list[str],
  },
  "instance":     str | None, #
  decision_augmentation.final_reference.instance
  "decision_guide": list | None, # decision_augmentation.signals_summary
  "total_score":  float,
}
```

Decision Guide 항목 구조:

```
{
  "level":        "knowledge" | "plan" | "instance",
  "failures":     list[str],
  "causes":       list[str],
  "corrections":  list[str],
}
```

3. proposal_planning 함수

3.1 함수 시그니처

```
def proposal_planning(
    example,          # GAIA 태스크 dict
    augmented_question: str, # 파일 설명 포함 확장 질문
```

```

model_name: str,
key: str,
url: str,
model: Any,
slm: bool,
retrieval_method: Callable,
top_k: int,
retrieval_option: str = "task_text", # "task_text" | "type_and_domain"
type_domain_retrieval_method = None,
plan_mode: str | None = None,
tools = None,
managed_agents = None,
planning_prompt_templates = None,
) -> dict

```

반환값:

```

{
  "plan": str, # "Knowledge:\n...\n\nPlan:\n...\n\nInstance:\n..."
  "knowledge": str, # Stage 1b 출력 (refined knowledge)
  "plan_steps": str, # Stage 2b 출력 (refined plan)
  "instance": str, # Stage 3b 출력 (refined instance)
  "retrieval_results": list,
  "examples": dict,
}

```

3.2 단계 [1] — 유사 태스크 검색

현재 질문과 가장 유사한 KB 레코드 top_k개를 가져와 few-shot 예시와 Decision Guide로 활용합니다.

```

# retrieval_option == "task_text" (기본)
retrieval_results = retrieval_method(example["question"], top_k=top_k)

# retrieval_option == "type_and_domain"
classification = classify_task_type_and_domain(task=augmented_question, ...)
retrieval_results = type_domain_retrieval_method(
    example["question"],
    task_types=classification["task_type_normalized"],
    domains=classification["domain_normalized"],
    top_k=top_k,
)

```

검색 후 6개 블록을 구성합니다:

```

knowledge_examples = build_similar_task_direction_blocks(retrieval_results)
plan_examples = build_similar_task_plan_blocks(retrieval_results)
instance_examples = build_instance_examples(retrieval_results)
guide_knowledge = build_decision_guide_blocks(retrieval_results,
level="knowledge")
guide_plan = build_decision_guide_blocks(retrieval_results,
level="plan")
guide_instance = build_decision_guide_blocks(retrieval_results,
level="instance")

```

3.3 Stage 1a — generate_knowledge

목적: 태스크를 풀기 위한 도메인 지식(선언적 + 절차적)을 생성합니다.

프롬프트 (`knowledge_prompt`):

```
Extract the domain knowledge required to understand and solve the given task.
Provide both:
    (a) declarative knowledge: domain concepts and definitions used
    (b) procedural knowledge: methodology, paradigm, or algorithm to apply
Return only the knowledge as plain text. Do NOT include the direct answer.

TASK: {{task}}
SIMILAR TASKS: {{examples}}
```

출력: `knowledge_draft`

3.4 Stage 1b — refine_knowledge

Decision Guide가 있으면 초안을 교정합니다. 없으면 초안을 그대로 반환합니다.

```
if not decision_guide:
    return draft
```

프롬프트 (`refine_knowledge_prompt`):

```
You generated domain knowledge for a task. A Decision Guide identifies
failure
patterns observed in similar tasks.
Refine the generated knowledge to address the failures. Only modify what the
guide says is insufficient or wrong.
Preserve all correct parts. Return the full refined knowledge as plain text.

TASK: {{task}}
GENERATED KNOWLEDGE: {{draft}}
DECISION GUIDE: {{decision_guide}}
```

출력: `knowledge_text`

3.5 Stage 2a — generate_plan

프롬프트 (`plan_prompt`):

```
Develop a high-level plan of no more than 10 steps to solve the given task
based only on the provided knowledge and similar tasks.
The plan must not include the final answer and the final conclusion.

Requirements:
- Use the knowledge explicitly and progressively in each step.
- Derive a decision criterion: an intermediate, task-specific quantity,
  rule, or structure that can be used to select the answer.

Strict Output Format:
- <step>
- <step>
- ...

TASK: {{task}}
KNOWLEDGE: {{knowledge}}
SIMILAR TASKS: {{examples}}
```

출력: `plan_draft`

3.6 Stage 2b — refine_plan

프롬프트 (`refine_plan_prompt`):

```
You generated a solution plan for a task. A Decision Guide identifies failure
patterns observed in similar tasks.
Refine the generated plan to address the failures. Only modify steps that the
guide says are wrong or missing.
Preserve all correct steps. Return the full refined plan as a bullet list.
```

```
TASK: {{task}}
KNOWLEDGE: {{knowledge}}
GENERATED PLAN: {{draft}}
DECISION GUIDE: {{decision_guide}}
```

출력: `plan_str`

3.7 Stage 3a — generate_instance

중간 계산값과 추론 과정을 포함하되 최종 답은 제외합니다.

프롬프트 (`instance_prompt`):

```
Generate a concrete execution instance that grounds the given plan into
specific, actionable steps with actual values and reasoning.
```

Requirements:

- Walk through each plan step with concrete values from the task.
- Show actual computations, lookups, or reasoning steps.
- Do NOT reveal or guess the final answer — stop just before the conclusion.

```
TASK: {{task}}
KNOWLEDGE: {{knowledge}}
PLAN: {{plan}}
SIMILAR INSTANCES: {{examples}}
```

출력: `instance_draft`

3.8 Stage 3b — refine_instance

프롬프트 (`refine_instance_prompt`):

```
You generated a concrete execution instance for a task. A Decision Guide
identifies failure patterns observed in similar tasks.
Refine the instance to fix the failures identified in the guide.
```

```
TASK: {{task}}
KNOWLEDGE: {{knowledge}}
PLAN: {{plan}}
GENERATED INSTANCE: {{draft}}
DECISION GUIDE: {{decision_guide}}
```

출력: `instance_str`

3.9 최종 plan 문자열 조합

```
final_plan = (
    f"Knowledge:\n{knowledge_text}\n\n"
    f"Plan:\n{plan_str}\n\n"
```

```
f"Instance:\n{instance_str}"
)
```

이 문자열이 `result["plan"]` 으로 반환되어 CodeAgent에 전달됩니다.

4. Generate-then-Refine 전략

4.1 흐름도

```
KB 유사 태스크 검색
├─ guide_knowledge = build_decision_guide_blocks(level="knowledge")
├─ guide_plan      = build_decision_guide_blocks(level="plan")
└─ guide_instance  = build_decision_guide_blocks(level="instance")

Stage 1a: generate_knowledge(task, examples)      → knowledge_draft
Stage 1b: refine_knowledge(draft, guide_knowledge) → knowledge_text
        └─ guide 없으면 draft 그대로 반환

Stage 2a: generate_plan(task, knowledge, examples) → plan_draft
Stage 2b: refine_plan(draft, guide_plan)           → plan_str
        └─ guide 없으면 draft 그대로 반환

Stage 3a: generate_instance(task, knowledge, plan, examples) → instance_draft
Stage 3b: refine_instance(draft, guide_instance)             → instance_str
        └─ guide 없으면 draft 그대로 반환
```

4.2 Decision Guide 신호 구조

```
# odysseuss_db: decision_augmentation.signals_summary
[
  {
    "level":      "knowledge" | "plan" | "instance",
    "failures":   ["실패 현상 ..."],
    "causes":     ["근본 원인 ..."],
    "corrections": ["수정 방법 ..."],
  },
  ...
]
```

`build_decision_guide_blocks()` 가 이를 프롬프트용 텍스트로 변환합니다:

```
[From Task: <task text>]
[KNOWLEDGE] Failure: ...
              Cause: ...
              Correction: ...
[PLAN] Failure: ...
...
```

4.3 Generate-then-Refine 방식을 선택한 이유

전략	특성
가이드와 함께 한 번에 생성	SLM은 긴 가이드 조건이 프롬프트 앞에 오면 컨텍스트 부담으로 초안 품질 저하
Generate-then-Refine	제약 없이 최선의 초안 생성 → 짧고 집중된 교정 프롬프트로 수정

5. CodeAgent 실행 — answer_plan 주입

5.1 proposal_planning 결과가 answer_plan으로 가는 경로

proposal 모드에서 `proposal_planning()` 결과는 `additional_knowledge`가 아닌 `answer_plan`으로 직접 삽입됩니다.

```
# run_gaia.py - proposal 모드 설정
agent.planner_fn = planner_fn          # planner_fn 콜백 등록
agent.run(augmented_question,
          additional_knowledge=None)    # None으로 전달

# smolagents/agents.py - planning_step() 내부
if self.planner_fn is not None:
    result = self.planner_fn(task, self.tools, self.managed_agents)
    proposal = result.get("plan", "")   # "Knowledge:\n...\n\nPlan:\n...\n\nInstance:\n..."

    if self.plan_mode in ("plan", "subtask", "plan_subtask",
                          "plan_subtask_action"):
        answer_plan = proposal          # ← 바로 answer_plan으로 사용

# answer_plan 이 PlanningStep 메모리에 기록됨
```

kb_type	경로
proposal	planner_fn 콜백 → result["plan"] → answer_plan (planning_step마다 호출)
plan_mode	plan_mode_planning() → additional_knowledge → LLM → answer_plan
original	KB 조회 → additional_knowledge → LLM → answer_plan

핵심: proposal 모드에서는 LLM이 plan을 재생성하지 않고, `proposal_planning()` 이 만든 "Knowledge+Plan+Instance" 블록이 그대로 `answer_plan` 이 됩니다.

5.2 에이전트 생성

```
manager_agent = CodeAgent(
    model=model,                      # TogetherAI SLM 또는 OpenAI
    max_steps=args.max_steps,         # 기본 12
    planning_interval=args.planning_interval, # 기본 1 (매 스텝마다 재계획)
    additional_authorized_imports=AUTHORIZED_IMPORTS,
    agent_kb=args.agent_kb,
    top_k=args.top_k,
    plan_mode=args.plan_mode,
)
```

5.3 ReAct 루프

```
Iteration 1 ~ max_steps:
┌ PlanningStep (planning_interval마다) ───────────────────────────────────┐
│ planner_fn() 호출 → answer_plan 갱신 │                                   │
└──────────────────────────────────────────────────────────────────────────┘

┌ ActionStep ─────────────────────────────────────────────────────────────────┐
│ Thought   : LLM이 다음 행동을 Python 코드로 작성 │                       │
│ Code      : Python 코드 실행 │                       │
│ Observation: 실행 결과(stdout/반환값) 수집 │                       │
└──────────────────────────────────────────────────────────────────────────┘
```

max_steps 초과 또는 final_answer() 호출 시 종료

5.4 사용 가능한 도구

도구	파일	역할
SearchTool	scripts/searcher.py	Tavily / Exa 웹 검색
CrawlerReadTool	scripts/async_web_crawler.py	URL 본문 크롤링
CrawlerArchiveSearchTool	scripts/async_web_crawler.py	Wayback Machine 검색
TextInspectorTool	scripts/text_inspector_tool.py	PDF/텍스트 검사
AudioInspectorTool	scripts/audio_inspector_tool.py	오디오 검사
VisualInspectorTool	scripts/ visual_inspector_tool.py	이미지 검사

6. 최종 답변 추출

6.1 prepare_response() — scripts/reformulator.py

```
def prepare_response(
    original_task: str,
    inner_messages,          # agent.write_memory_to_messages() 결과
    reformulation_model: Model,
) -> str
```

처리 과정:

1. 시스템 메시지에 원래 태스크 제시
2. 에이전트 메모리를 USER 역할로 변환하여 대화 이력 구성
3. 최종 지시 메시지 추가

포맷 규칙:

- 숫자: 숫자만, 쉼표·단위 제거 (질문에서 요구하면 유지)
- 텍스트: 관사·약어 제거, 마침표 제외
- 목록: 쉼표로 구분
- 답 불명: "Unable to determine"

1. `response.split("FINAL ANSWER: ")[-1].strip()` 으로 답 추출

7. 데이터 스키마

7.1 odysseuss_db.jsonl 레코드

```
{
  "task_id": "string",
  "task": "string",
  "true_answer": "string",
  "agent_planning": "string | null",
}
```

```

"task_analysis": {
  "knowledge": "string",
  "plan": ["step1", "step2", "..."],
  "task_type": ["raw_value", "Normalized Value"],
  "domain": ["raw_value", "Normalized Value"],
  "task_type_normalized": ["Normalized Value"],
  "domain_normalized": ["Normalized Value"]
},
"decision_augmentation": {
  "final_reference": {
    "knowledge": "string",
    "plan": ["step1", "..."],
    "instance": "string"
  },
  "signals_summary": [
    {
      "level": "knowledge | plan | instance",
      "failures": ["string"],
      "causes": ["string"],
      "corrections": ["string"]
    }
  ]
}
}

```

7.2 proposal_planning() 반환값

```

{
  "plan": str,
  # "Knowledge:\n{knowledge_text}\n\nPlan:\n{plan_str}\n\nInstance:\n{instance_str}"
  # → CodeAgent.planning_step()에서 answer_plan 으로 직접 사용

  "knowledge": str, # Stage 1b 출력
  "plan_steps": str, # Stage 2b 출력
  "instance": str, # Stage 3b 출력
  "retrieval_results": list, # KB 검색 결과 원본
  "examples": dict,
}

```

부록. 모듈 파일 경로

역할	파일 경로
전체 실행 진입점	examples/open_deep_research/run_gaia.py
KB 인덱싱·검색	examples/open_deep_research/agent_kb/agent_kb_retrieval.py
KB 서비스	examples/open_deep_research/agent_kb/agent_kb_service.py
Proposal Planning	examples/open_deep_research/planner_kb/planner.py
프롬프트 템플릿	examples/open_deep_research/planner_kb/planner_prompts.yaml
Planner 공개 API	examples/open_deep_research/planner_kb/__init__.py
최종 답변 추출	examples/open_deep_research/scripts/reformulator.py
모델 헬퍼	examples/open_deep_research/scripts/automodel.py